

API Security Risk Analysis

Technical Assessment Report | Task 3

Prepared by

Harsh Aghera

Internship

Future Interns – Cyber Security

Task

Task 3 – API Security Risk Analysis

Date

July 17, 2026

Repository

https://github.com/Harshaghera111/FUTURE_CS_03

Table of Contents

Note: After opening in Microsoft Word, right-click the field below and select "Update Field" to refresh page numbers.

1. Executive Summary

This report presents the findings of an API security risk analysis conducted as part of the Future Interns Cyber Security Internship – Task 3. The assessment targeted the publicly available JSONPlaceholder REST API (<https://jsonplaceholder.typicode.com>), which is used strictly for educational and demonstration purposes. No production systems or sensitive data were involved in this assessment.

The objective was to evaluate the API against common security risks defined by the OWASP API Security Top 10 framework and to document observations, evidence, and recommendations in a structured format consistent with professional penetration testing reports.

This report presents the findings of a manual API security risk analysis conducted against the JSONPlaceholder REST API (<https://jsonplaceholder.typicode.com>) as part of the Future Interns Cyber Security Internship – Task 3. Testing was performed using Postman across eight endpoints on 17 July 2026.

Seven findings were identified: zero Critical, zero High, zero Medium, two Low, and five Informational. The two Low findings relate to absent HTTP security response headers (F-04) and the pattern of verbose user data fields returned in API responses (F-03). Five Informational findings document authentication architecture, authorization controls, rate limiting presence, error handling behaviour, and input validation scope limitations.

The overall security posture of JSONPlaceholder is appropriate for a publicly accessible demonstration API with entirely fictional data. The primary actionable observation is the absence of standard HTTP security response headers, which would be a mandatory baseline requirement in any production deployment. All findings and recommendations in this report serve as a professional framework for evaluating equivalent controls in production environments.

2. Objectives

- Identify and document API security weaknesses in the target API.
- Evaluate the API against the OWASP API Security Top 10 (2023 edition).
- Assess HTTP response headers for security-relevant misconfigurations.
- Analyse error handling behaviour for information disclosure risks.
- Review authentication and authorisation controls.
- Provide actionable, risk-rated remediation recommendations.
- Produce a professional deliverable suitable for a real-world security engagement.

3. Scope

3.1 In Scope

- Base URL: <https://jsonplaceholder.typicode.com>
- Endpoints: /users, /posts, /comments, /albums, /photos, /todos
- HTTP methods: GET (primary), with structural review of simulated POST/PUT/PATCH/DELETE
- Response headers and body analysis
- Error handling review (4xx responses)
- Security header assessment

3.2 Out of Scope

- Production APIs or any live systems not explicitly listed above
- Denial-of-Service (DoS) or load testing
- Authentication bypass or exploitation attempts
- Network-layer or infrastructure testing
- Client-side (browser/mobile) application testing

3.3 Engagement Constraints

- This assessment is read-only and non-destructive.
- No actual data was created, modified, or deleted on the target API.
- JSONPlaceholder simulates responses; no real user data exists.
- All testing was performed in accordance with the internship guidelines.

4. Target API Overview

API Name	JSONPlaceholder REST API
Base URL	https://jsonplaceholder.typicode.com
Purpose	Free, open-source fake REST API for testing and prototyping
Maintainer	Typicode
API Type	REST (Representational State Transfer)
Data Format	JSON (application/json)
Authentication	None – publicly accessible, no API key required
Authorization	None – all endpoints are world-readable
Protocol	HTTPS (TLS)

4.1 Endpoints Tested

#	Endpoint	Method	Description	Response Code
1	/users	GET	Retrieve list of all users	200 OK
2	/posts	GET	Retrieve list of all posts	200 OK
3	/comments	GET	Retrieve list of all comments	200 OK
4	/albums	GET	Retrieve list of all albums	200 OK
5	/photos	GET	Retrieve list of all photos	200 OK
6	/todos	GET	Retrieve list of all todos	200 OK
7	/invalidendpoint	GET	Non-existent endpoint (error test)	404 Not Found
8	/users/999	GET	Non-existent user ID (error test)	404 Not Found

5. Testing Environment

Operating System	Windows 11
Primary Tool	Postman (API Client)
Testing Method	Manual black-box API assessment
Network	Standard internet connection – no proxy interception
Assessment Period	July 17, 2026
Tester	<i>Harsh Aghera</i>

5.1 Ethical Statement

All testing activities described in this report were conducted in a strictly controlled, read-only, and non-disruptive manner against a publicly available demonstration API. No exploits, destructive payloads, or automated scanning tools were used. This assessment was performed solely for educational purposes as part of the Future Interns Cyber Security Internship programme. The tester adhered to responsible disclosure principles and the ethical guidelines of the internship at all times.

6. Methodology

The assessment followed a structured, multi-phase methodology aligned with industry best practices and the OWASP API Security Testing Guide. Each phase is described below.

6.1 Information Gathering

Reconnaissance was conducted on the target API using publicly available documentation. The base URL, available endpoints, HTTP methods, and data formats were catalogued prior to testing.

- Review official JSONPlaceholder documentation
- Enumerate all accessible endpoints
- Identify HTTP methods supported per endpoint
- Document the authentication model (or lack thereof)

6.2 API Request Analysis

Each endpoint was queried using Postman to capture the full HTTP request and response cycle. Requests were crafted manually to ensure controlled and repeatable tests.

- Send GET requests to all in-scope endpoints
- Capture and save raw request and response data
- Screenshot Postman output for evidence documentation

6.3 Response Analysis

API responses were analysed for content, structure, and any sensitive data fields that would represent a risk in a production equivalent.

- Review response body for PII or sensitive fields
- Verify content-type headers match actual response format
- Assess verbosity of response data

6.4 HTTP Header Inspection

HTTP response headers were inspected for the presence, absence, or misconfiguration of security-relevant headers.

- Capture full response header set from /users endpoint
- Check for: Content-Security-Policy, X-Frame-Options, X-Content-Type-Options, Strict-Transport-Security, Referrer-Policy, Permissions-Policy

- Document missing or weak security headers

6.5 Error Handling Review

Intentionally invalid requests were submitted to evaluate how the API handles errors and whether error responses disclose sensitive information.

- Request a non-existent endpoint (/invalidendpoint)
- Request a non-existent resource (/users/999)
- Analyse error response body and status codes

6.6 Security Header Review

A dedicated review was performed on security-relevant HTTP headers to identify misconfigurations that would be exploitable in a production environment.

- Cross-reference observed headers against OWASP recommendations
- Identify absent security headers
- Assess cache-control and CORS policy headers

6.7 Risk Assessment Approach

Each finding was rated using a qualitative risk matrix based on Likelihood × Impact. Risk ratings follow the OWASP risk methodology.

- Critical – Immediate exploitation risk, severe business impact
- High – Significant risk, likely to be exploited
- Medium – Moderate risk, exploitation requires conditions
- Low – Minor risk, limited impact
- Informational – No direct risk; noted for awareness

7. Findings

The following findings were identified during the assessment. Each finding includes an observation, supporting evidence, risk rating, business impact, and recommendation. Findings are organised by category consistent with the OWASP API Security Top 10 framework.

⚠ Note: JSONPlaceholder is a public demonstration API with no production intent. Where a finding would only be applicable to a real production API, this is explicitly stated in the Observation.

9.1 Authentication Controls

Observation	All eight tested endpoints responded with HTTP 200 OK without requiring any form of authentication credential. No API key, Bearer token, session cookie, or HTTP Basic Authentication header was present in any request. The Postman Authorization tab was configured to 'No Auth' for all requests, confirming the API enforces no authentication mechanism. This behaviour is intentional for JSONPlaceholder, a publicly available demonstration API containing entirely fictional data. In a production environment, unauthenticated access to equivalent endpoints would constitute a Critical vulnerability under OWASP API Security Top 10 – API1:2023 (Broken Authentication).
Evidence	GET_Users_Request_Response.png, GET_Posts_Request_Response.png, GET_Comments_Request_Response.png, GET_Albums_Request_Response.png, GET_Photos_Request_Response.png, GET_Todos_Request_Response.png
Risk Level	Informational
Business Impact	For JSONPlaceholder as a demonstration API, the absence of authentication carries no real business impact as all data is entirely fictitious. In a production-equivalent system, unrestricted unauthenticated access would expose all user records, enabling mass data enumeration, account discovery, and potential violations of applicable data protection legislation.
Recommendation	For production API implementations, enforce OAuth 2.0 with JWT Bearer token authentication or API key authentication enforced at the gateway layer. All endpoints must validate a credential before returning data. Unauthenticated requests should receive HTTP 401 Unauthorized with an appropriate WWW-Authenticate response header.

9.2 Authorization Controls

Observation	No authorization controls were observed across any tested endpoint. All resources were accessible to any client without identity verification or ownership checks. Individual user resources could be retrieved by numeric ID (e.g., GET /users/1) without any mechanism to confirm the requesting client's entitlement to that specific record. This behaviour is consistent with the public nature of JSONPlaceholder and is not a defect in this context. In a production API, this pattern would constitute a Broken Object Level Authorization (BOLA) vulnerability – OWASP API Security Top 10 – API1:2023, consistently ranked as the most prevalent and impactful API security weakness.
--------------------	--

Evidence	GET_Users_Request_Response.png, GET_Users_Response.png
Risk Level	Informational
Business Impact	For this demonstration API, no business impact exists as all data is fictional. In a production API, the absence of object-level authorization would permit any client to access records belonging to other users by enumerating sequential resource IDs, representing a severe and easily exploitable data breach risk.
Recommendation	Production APIs must implement per-object authorization checks that verify the requesting user's entitlement to each specific resource before returning data. Authorization logic must not rely on the obscurity of resource IDs. Return HTTP 403 Forbidden for unauthorized access attempts and log all such events for audit review.

9.3 Sensitive Data Exposure

Observation	The GET /users endpoint returns a comprehensive fictional user profile for each record including: full name, username, email address, full postal address (street, suite, city, zip code), geographical coordinates (latitude and longitude), phone number, personal website URL, and company details including name, catch phrase, and business tagline. The GET /comments endpoint additionally returns email addresses associated with each comment entry. All data returned by JSONPlaceholder is entirely synthetic. However, the response schema demonstrates a pattern of excessive data exposure with no field-level filtering or data minimisation applied. The response header x-powered-by: Express also discloses the underlying application framework, which could assist targeted exploit selection in a production context.
Evidence	GET_Users_Response.png, GET_Users_Request_Response.png, GET_Comments_Request_Response.png, GET_Users_Headers.png
Risk Level	Low
Business Impact	For this demonstration API, actual business impact is nil as no real personal data is exposed. In a production environment, unnecessary exposure of PII fields such as email addresses, full postal addresses, and geolocation coordinates could facilitate social engineering attacks, targeted phishing campaigns, physical location tracking, and violations of data protection regulations including GDPR.
Recommendation	Production APIs should apply response field filtering to return only data fields required by the consuming application. Exclude geolocation coordinates, full postal addresses, and internal company metadata from general list endpoints unless explicitly required by a documented use case. Remove the x-powered-by header from all responses to prevent server technology fingerprinting.

9.4 Security Headers

Observation	Inspection of the HTTP response headers for GET /users (26 headers total) identified that several security-relevant headers recommended by OWASP and industry best practice were absent from the API response. Headers observed: x-content-type-options: nosniff, cache-control: max-age=43200 (12-hour client caching), pragma: no-cache, content-type: application/json; charset=utf-8, etag: W/"160d-1eMSsxeJRfnVLRBmYJSbCiJZ1qQ". Headers absent from all tested responses: Content-Security-Policy (CSP), X-Frame-
--------------------	--

	Options, Strict-Transport-Security (HSTS), Referrer-Policy, Permissions-Policy. The x-powered-by: Express header discloses the backend application framework.
Evidence	GET_Users_Headers.png
Risk Level	Low
Business Impact	Missing security headers increase the residual attack surface of web applications consuming this API. The absence of HSTS could permit SSL stripping on unprotected connections. Missing Content-Security-Policy increases XSS exploitation risk in browser-facing frontend applications. Absent X-Frame-Options could enable clickjacking. Technology disclosure via x-powered-by assists targeted exploit selection against known Express.js vulnerabilities.
Recommendation	Configure the API server or reverse proxy to include on all responses: Strict-Transport-Security: max-age=31536000; includeSubDomains Content-Security-Policy: default-src 'none' X-Frame-Options: DENY Referrer-Policy: no-referrer Permissions-Policy: geolocation=(), microphone=(), camera=() Remove the x-powered-by header entirely. These settings are typically applied at the infrastructure layer (Nginx, Cloudflare, API Gateway) and require no application code changes.

9.5 Rate Limiting

Observation	The response header x-ratelimit-limit: 1000 was observed in the HTTP response headers for GET /users, indicating a rate limiting mechanism is implemented with a threshold of 1,000 requests. However, the rate limiting time window (per second, minute, or hour) was not disclosed in the captured response headers. Supplementary headers indicating remaining allowance (x-ratelimit-remaining) and reset timestamp (x-ratelimit-reset) were not visible in the response. No rate limit enforcement was triggered during testing as only a small number of manual requests were submitted. Formal load or stress testing was outside the scope of this assessment. The effectiveness of the rate limiting mechanism cannot be fully verified within the scope of this engagement.
Evidence	GET_Users_Headers.png
Risk Level	Informational
Business Impact	The presence of an x-ratelimit-limit header is a positive indicator that denial-of-service and enumeration risks have been considered. Without confirmation of the enforcement window and mechanism, residual risk cannot be precisely quantified. In production, inadequate rate limiting could enable denial-of-service attacks, credential stuffing, and mass data enumeration at scale.
Recommendation	Verify and expose the full rate limiting configuration in response headers, including the time window (x-ratelimit-window), remaining request count (x-ratelimit-remaining), and reset timestamp (x-ratelimit-reset). Ensure limits are enforced per authenticated identity or source IP for unauthenticated endpoints. Return HTTP 429 Too Many Requests with a Retry-After header when limits are exceeded.

9.6 Error Handling

Observation	Two deliberate error scenarios were tested to evaluate the API's error handling behaviour. A request to a non-existent endpoint (GET /invalidendpoint) returned HTTP 404 Not Found with a response body of {} (empty JSON object, 1.08 KB). A request for a non-existent user resource (GET /users/999) also returned HTTP 404 Not Found with a response body of {} (1.07 KB). Neither response included stack traces, internal error codes, file system paths, database query details, framework names, or any sensitive internal information. Error responses were minimal, correctly formatted as JSON, and consistent with the API content-type declaration. This represents a positive security posture for error handling and information disclosure prevention.
Evidence	Invalid_Endpoint.png, User_Not_Found.png
Risk Level	Informational
Business Impact	Proper minimal error handling prevents information leakage that could assist an attacker in mapping the internal architecture, framework, or data model of an application. The current error handling behaviour is appropriate, consistent with OWASP secure coding guidelines, and represents no current risk.
Recommendation	Maintain the current minimal error response approach in production implementations. Ensure all error responses return generic, non-verbose messages and never expose stack traces, internal class names, database query fragments, or file system paths. Consider adopting RFC 9457 (Problem Details for HTTP APIs) for standardised, consumer-friendly error responses without disclosing internal implementation details.

9.7 Input Validation

Observation	Input validation testing was limited to read-only GET requests within the agreed scope of this assessment. No POST, PUT, PATCH, or DELETE requests with crafted payloads were submitted, as write operations were outside the defined scope. All well-formed GET requests returned expected JSON responses with HTTP 200 OK. One notable observation: the endpoint URL /Comments (with an uppercase 'C') returned HTTP 200 OK with 40.77 KB of data, suggesting the server applies case-insensitive URL routing. This is not a vulnerability in isolation but is documented for completeness. No SQL injection, path traversal, parameter pollution, or fuzzing payloads were tested. Input validation robustness cannot be verified within the scope of this assessment.
Evidence	GET_Comments_Request_Response.png
Risk Level	Informational
Business Impact	Unable to fully assess within the current read-only scope. In production APIs, insufficient input validation could enable injection attacks, path traversal, mass assignment vulnerabilities, and parameter pollution – OWASP API Security Top 10 – API3:2023 (Broken Object Property Level Authorization) and API8:2023 (Security Misconfiguration).
Recommendation	Production APIs must implement server-side input validation for all user-supplied inputs including path parameters, query strings, and request bodies. Prefer allowlisting over denylisting. Validate data types, lengths, formats, and value ranges. Return HTTP 400 Bad Request for invalid input without disclosing internal schema details or validation logic. Conduct a dedicated input validation assessment in a follow-up engagement.

8. Risk Assessment Matrix

The table below summarises all findings by risk level. Populate this matrix after completing Section 7.

#	Finding	Category	Likelihood	Impact	Risk Level	Status
F-01	Authentication Controls	API1:2023	Low	High	Informational	Open
F-02	Authorization Controls	API1:2023	Low	High	Informational	Open
F-03	Sensitive Data Exposure	API3:2023	Low	Medium	Low	Open
F-04	Security Headers	API7:2023	Medium	Low	Low	Open
F-05	Rate Limiting	API4:2023	Low	Medium	Informational	Open
F-06	Error Handling	API3:2023	Low	Low	Informational	Open
F-07	Input Validation	API8:2023	Low	Low	Informational	Open

8.1 Risk Level Legend

Risk Level	Colour Code	Description
Critical	#FF0000	Immediate threat; exploitation likely; severe impact
High	#E74C3C	Significant risk; exploitable with moderate effort
Medium	#F39C12	Moderate risk; conditional exploitation
Low	#27AE60	Minimal risk; limited real-world impact
Informational	#2980B9	No direct risk; noted for completeness

9. Recommendations

This section will consolidate actionable remediation guidance for all findings identified in Section 7. Recommendations are organised by priority (Critical → Informational).

9.1 Critical & High Priority

No Critical or High risk findings were identified during this assessment. All tested endpoints returned appropriate responses and no exploitable vulnerabilities were confirmed within the scope of the engagement. No immediate critical remediation actions are required for the JSONPlaceholder demonstration API in its current form.

9.2 Medium Priority

F-04 – Security Headers (Low / Medium in production context):

Although rated Low for this demonstration API, the absence of standard HTTP security response headers would represent a Medium risk in any production deployment.

Configure the API server, reverse proxy, or CDN to include the following on all responses:

- Strict-Transport-Security: max-age=31536000; includeSubDomains
- Content-Security-Policy: default-src 'none'
- X-Frame-Options: DENY
- Referrer-Policy: no-referrer
- Permissions-Policy: geolocation=(), microphone=(), camera=()
- Remove the x-powered-by header entirely

These configurations are typically applied at the infrastructure layer (Nginx, Cloudflare Workers, AWS CloudFront, API Gateway) and require no application code changes.

9.3 Low & Informational

F-03 – Sensitive Data Exposure (Low):

- Review all API response schemas and implement field-level response filtering.
- Return only data fields required by the consuming application.
- Exclude geolocation coordinates, full postal addresses, and company metadata from general list endpoints unless explicitly required.

F-05 – Rate Limiting (Informational):

- Expose the full rate limiting configuration in response headers: x-ratelimit-remaining, x-ratelimit-reset, x-ratelimit-window.

- Document and verify enforcement behaviour under load conditions.

F-06 – Error Handling (Informational):

- Maintain the current minimal error response approach.
- Standardise using RFC 9457 Problem Details format for production deployments.

F-07 – Input Validation (Informational):

- Establish server-side input validation for all input types in production.
- Conduct a dedicated input validation engagement as a follow-up assessment.

9.4 General Security Hardening

The following hardening measures are recommended for any production API equivalent to the system assessed in this engagement:

Authentication: Implement OAuth 2.0 with JWT Bearer token authentication. All endpoints must require a valid, unexpired token before returning data.

Authorization: Implement object-level authorization (BOLA prevention) verifying per-request resource ownership before data is returned or modified.

API Gateway: Deploy an API Gateway or WAF enforcing rate limiting, threat detection, and request validation policies at a centralised infrastructure layer.

Logging & Monitoring: Implement structured API access logging. Alert on anomalous request patterns, elevated error rates, and rate limit breaches.

SDLC Integration: Incorporate automated API security testing (OWASP ZAP, Burp Suite) into the CI/CD pipeline as part of a DevSecOps practice.

Security Reviews: Conduct periodic security assessments against the OWASP API Security Top 10 (2023 Edition) as the API evolves and new endpoints are introduced.

10. Conclusion

This report presents the complete API security risk assessment conducted against the JSONPlaceholder REST API as part of the Future Interns Cyber Security Internship – Task 3. The engagement was performed following a structured methodology aligned with the OWASP API Security Top 10 and industry best practices.

This assessment evaluated the JSONPlaceholder REST API across eight endpoints using manual black-box testing via Postman on 17 July 2026. Seven findings were identified in total: zero Critical, zero High, zero Medium, two Low, and five Informational.

The two Low findings document the absence of standard HTTP security response headers (F-04) and the pattern of verbose user data fields in API responses (F-03). The five Informational findings cover: the absence of authentication by design (F-01), the absence of authorization controls by design (F-02), the presence of rate limiting headers with unconfirmed enforcement parameters (F-05), positive minimal error handling behaviour (F-06), and input validation which could not be fully assessed within the read-only scope (F-07).

The overall security posture of JSONPlaceholder is appropriate for a publicly accessible demonstration API with entirely fictional data. Error handling behaviour was positive, returning minimal, non-verbose responses with no information leakage. Rate limiting indicators were observed in response headers. The primary area for improvement is the addition of standard HTTP security response headers, which can be implemented at the infrastructure level with minimal engineering effort.

All findings and recommendations in this report should be evaluated in the context of a production system, where authentication, authorization, data minimisation, and security header configuration are mandatory baseline security requirements for any externally accessible API.

This assessment was conducted in an ethical and non-disruptive manner. All findings and recommendations are intended solely for educational purposes and professional skill development.

11. References

[1]	OWASP API Security Top 10 – 2023 Edition	https://owasp.org/API-Security/editions/2023/en/0x00-header/
[2]	JSONPlaceholder – Free Fake REST API	https://jsonplaceholder.typicode.com
[3]	RFC 9110 – HTTP Semantics	https://www.rfc-editor.org/rfc/rfc9110
[4]	RFC 9112 – HTTP/1.1	https://www.rfc-editor.org/rfc/rfc9112
[5]	OWASP API Security Testing Guide	https://owasp.org/www-project-web-security-testing-guide/
[6]	OWASP Secure Headers Project	https://owasp.org/www-project-secure-headers/
[7]	MDN Web Docs – HTTP Headers Reference	https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers
[8]	Postman API Platform Documentation	https://learning.postman.com/docs/
[9]	NIST Special Publication 800-95 – Guide to Secure Web Services	https://csrc.nist.gov/publications/detail/sp/800-95/final

All URLs verified as of July 17, 2026. Access may require an active internet connection.